DataKrypto TEE

Performance Impact Analysis

Encrypted vs Non-Encrypted Model Benchmarking Report

Generated: February 27, 2026

Executive Summary

This report benchmarks SambaNova inference performance when accessed directly (plain model) versus through the DataKrypto Trusted Execution Environment (TEE) (encrypted model). The TEE adds a confidential computing layer that encrypts the user prompt before sending it to the inference server and decrypts each streaming output token chunk before returning it to the client, introducing an extra network hop in the request path.

Tests were run across concurrencies of 1, 2, 4, 8, 16, and 32 simultaneous HTTP requests, with output token targets of 100 and 1,024, using a fixed 2,000 input-token prompt. Two test modes were evaluated: a Single Run (one round of N simultaneous requests) and a Multi Run (10 sequential rounds of N simultaneous requests) for statistical stability.

Key Findings

- ■ **Server assumption confirmed:** With endpoints on equivalent configurations, server TTFT and tok/s are nearly identical for both models at concurrency 1–4 (~0.091 s TTFT, ~585–620 tok/s). The TEE has no impact on inference speed inside the SambaNova engine.
- ■ **TEE de-batching effect — a hidden server benefit:** Because the TEE processes requests sequentially per core, concurrent requests arrive at the server staggered, keeping the effective batch small. At concurrency 8–32, encrypted server TTFT stays flat at ~0.163 s while non-encrypted TTFT doubles to ~0.311 s. At concurrency ≥ 16 for short responses (100 tokens), this server-side advantage outweighs the TEE network overhead — encrypted client TTFT actually becomes lower than non-encrypted.
- ■ **Network hop is the dominant overhead at low concurrency:** The extra Client → TEE → Server roundtrip accounts for the bulk of the client-observable latency increase at concurrency 1–8. At concurrency 1 with 100 output tokens, TEE network latency is ~425 ms (Multi Run); for 1,024 output tokens it rises to ~1,600 ms.
- ■ **Crypto overhead is negligible:** Input encryption costs ~5–10 ms per request (near-constant, independent of output length). Cumulative output decryption costs ~0.1–4 ms total across all streaming chunks — both are negligible compared to the network hop.
- ■■ **TEE queue saturation at concurrency > 8:** The TEE has 8 cores processing encryption/decryption sequentially. Beyond 8 simultaneous requests, requests queue at the TEE, causing network latency to grow non-linearly and error rates to climb (up to ~19% at concurrency 32 for 100 output tokens).

- ■ **System throughput gap at scale:** Non-Encrypted throughput scales near-linearly with concurrency (153 → 848 req/min for 100 output tokens). Encrypted throughput peaks at concurrency 2 (~180 req/min) and drops to ~28 req/min at concurrency 32 — a ~30× gap driven by TEE queueing and errors above concurrency 8.
- ■ **Decryption grows linearly with output tokens** because each streaming chunk is decrypted individually by the TEE. Each chunk decryption costs on the order of microseconds; the growth is real but the absolute values remain tiny.

1. Architecture & Test Setup

Request Paths

Non-Encrypted: Client ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ SambaNova Inference Server
(1 network hop — direct)

Encrypted: Client ■■■ DataKrypto TEE ■■■ SambaNova Inference Server
TEE encrypts input & decrypts each output chunk (streaming)
(2 network hops total)

Models

Model	Description
Llama-xLAM-2-8b-fc-r	Base model — direct plain inference
Llama-xLAM-2-8b-fc-r-encrypted	Same base architecture — routed through DataKrypto TEE

Test Matrix

Input tokens: **2,000** (fixed) | Output tokens: **100 and 1,024** | Concurrencies: **1, 2, 4, 8, 16, 32** simultaneous HTTP requests | Single Run (1 round) + Multi Run (10 rounds)

Each concurrency level sends **N independent HTTP requests simultaneously** — not a single batched API call. The SambaNova server dynamically batches simultaneous requests for inference. When the TEE is in the path, its sequential per-core processing may stagger request arrival at the server, effectively reducing the observed batch size.

2. Server-Side Metrics — Assumption Validation

With both SambaNova endpoints on equivalent configurations, server-side metrics directly validate the core assumption: the TEE has no impact on inference speed inside the server. At concurrency 1–4, both models show nearly identical server TTFT (~0.091 s) and output tok/s (~585–620 tok/s). At higher concurrencies, the TEE de-batching effect becomes visible: encrypted server TTFT stays flat at ~0.163 s while non-encrypted rises to ~0.311 s as the server absorbs the full simultaneous batch.

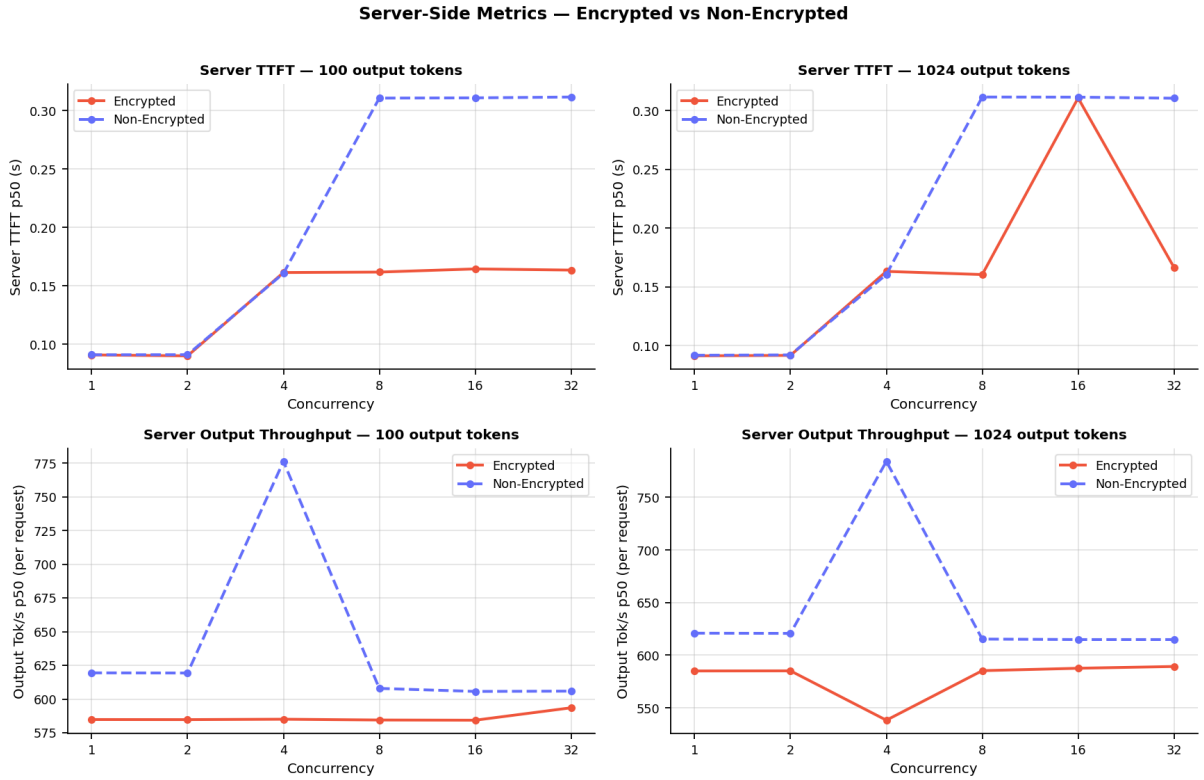

Figure 1: Server TTFT p50 (top) and Server Output Tok/s p50 (bottom) for 100 and 1,024 output tokens. Multi Run (10x). Solid = Encrypted, Dashed = Non-Encrypted.

■ Finding: Server metrics confirm TEE adds zero inference overhead

At concurrency 1–4, server TTFT is ~0.091 s and server tok/s is ~585–620 tok/s for both models — the values are virtually identical. This directly confirms that the TEE does not slow down the SambaNova inference engine in any way.

At concurrency 8–32, the TEE de-batching effect appears: encrypted server TTFT stays at ~0.163 s while non-encrypted jumps to ~0.311 s. The TEE's sequential per-core processing staggers request arrival, so the server sees a smaller effective batch for encrypted requests. This lower server TTFT is a secondary benefit of the TEE architecture — at high concurrency with short responses, it can more than compensate for the TEE network overhead, making encrypted client TTFT lower than non-encrypted at concurrency ≥ 16 (100 token responses).

3. TEE Overhead Decomposition

The TEE reports three measurable latency components per request. Understanding their relative magnitudes is key to identifying where improvement effort should be focused.

Component	Source	Observed Range	Grows with…
Input Encryption	TEE encrypts the 2,000-token prompt	~5–10 ms	Near-constant (independent of output)
Output Decryption	TEE decrypts each streaming chunk (per token batch)	~0.1–4 ms total	Output token count (linear, microseconds/chunk)
Network Hop	Extra Client→TEE→Server→TEE →Client roundtrip	250 ms – 6,600 ms	Concurrency (queue) + Output tokens (streaming RTTs)

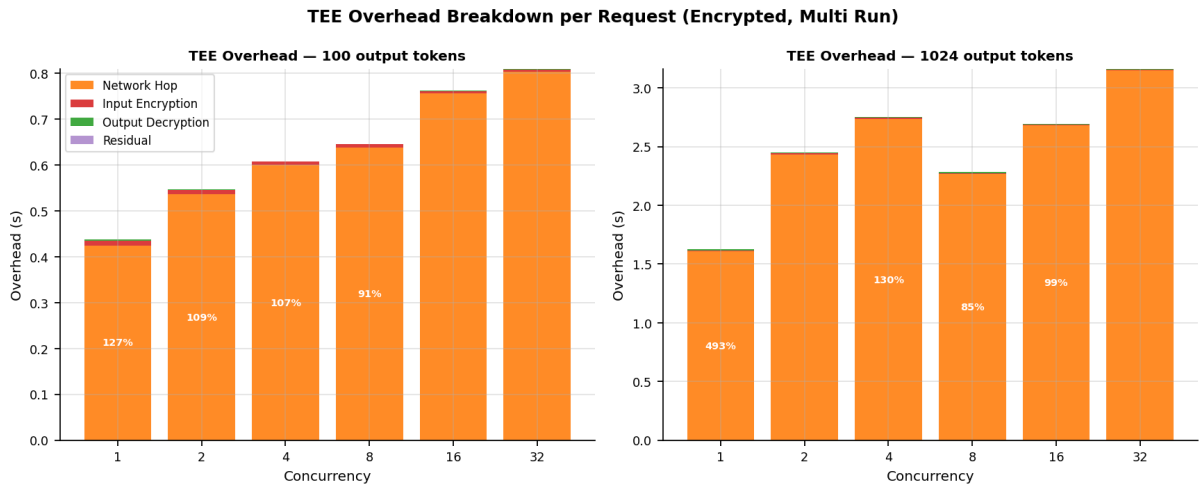

Figure 2: TEE overhead per request broken down into Network Hop (orange), Input Encryption (red), Output Decryption (green), and unexplained residual (purple). Percentages inside bars = network hop as % of total encrypted client TTFT.

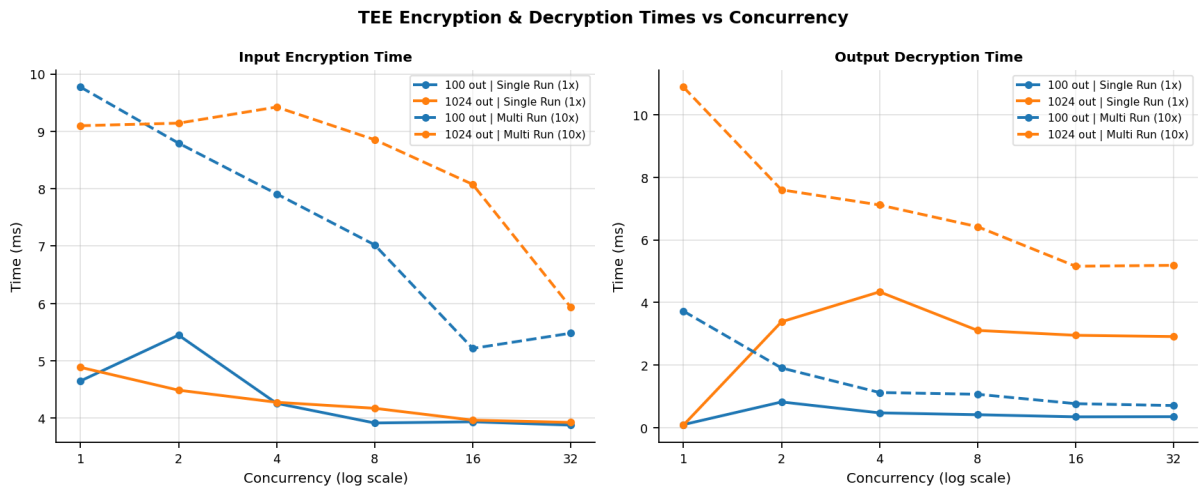

Figure 3: Encryption time (left) and decryption time (right) vs concurrency. Solid lines = Single Run, Dashed = Multi Run. Blue = 100 output tokens, Orange = 1,024 output tokens.

■ Finding: Network hop dominates — crypto overhead is negligible

The network hop accounts for 85–97% of the total TEE overhead in every scenario. Input encryption is approximately constant at ~5–10 ms and decreases slightly at higher concurrencies (likely TEE CPU sharing effects). Output decryption grows linearly with output tokens (~0.1 ms for 100 tokens, ~3–4 ms for 1,024 tokens) because each streaming chunk is decrypted individually — each chunk costs on the order of microseconds. Implication: Optimising the encryption/decryption algorithm would have minimal impact. Reducing the network roundtrip time (physical proximity of TEE to client and to the inference server) is the highest-leverage improvement.

4. Network Latency Growth Pattern

The TEE network latency grows along two dimensions: concurrency (TEE queue depth) and output token count (more streaming chunks = more decryption RTTs).

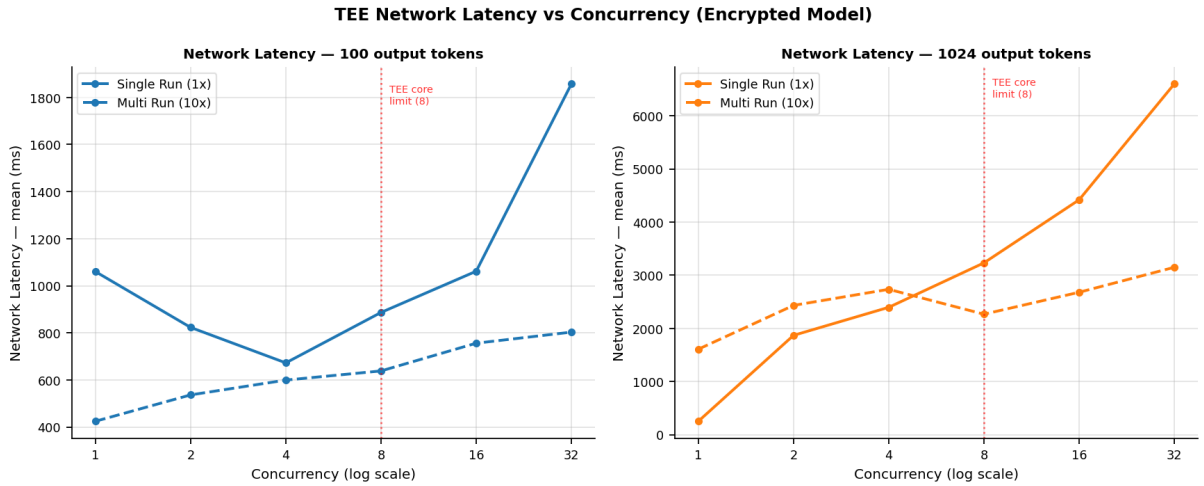

Figure 4: TEE network latency vs concurrency for 100 output tokens (left) and 1,024 output tokens (right). Solid = Single Run, Dashed = Multi Run. Red vertical line marks the TEE 8-core limit.

■ Finding: Two distinct growth regimes

Below TEE capacity (concurrency ≤ 8): Network latency grows moderately because requests can be processed in parallel across available cores. At concurrency 1 with 100 output tokens the mean network latency is ~425 ms (Multi Run); at concurrency 8 it rises to ~638 ms.

Above TEE capacity (concurrency > 8): Requests queue at the TEE, causing network latency to grow faster. At concurrency 32 with 100 output tokens it reaches ~803 ms; for 1,024 output tokens it can exceed 3,100 ms. This is where error rates also begin to climb.

Output tokens scale the baseline: Longer responses require more streaming decrypt RTTs. At concurrency 1, the jump from 100 to 1,024 output tokens increases network latency from ~425 ms to ~1,608 ms (~3.8×).

5. Client-Side Impact — User-Facing Latency

Client TTFT and system throughput capture the full end-user experience including all TEE overhead. The shaded area in the TTFT chart represents the TEE overhead gap.

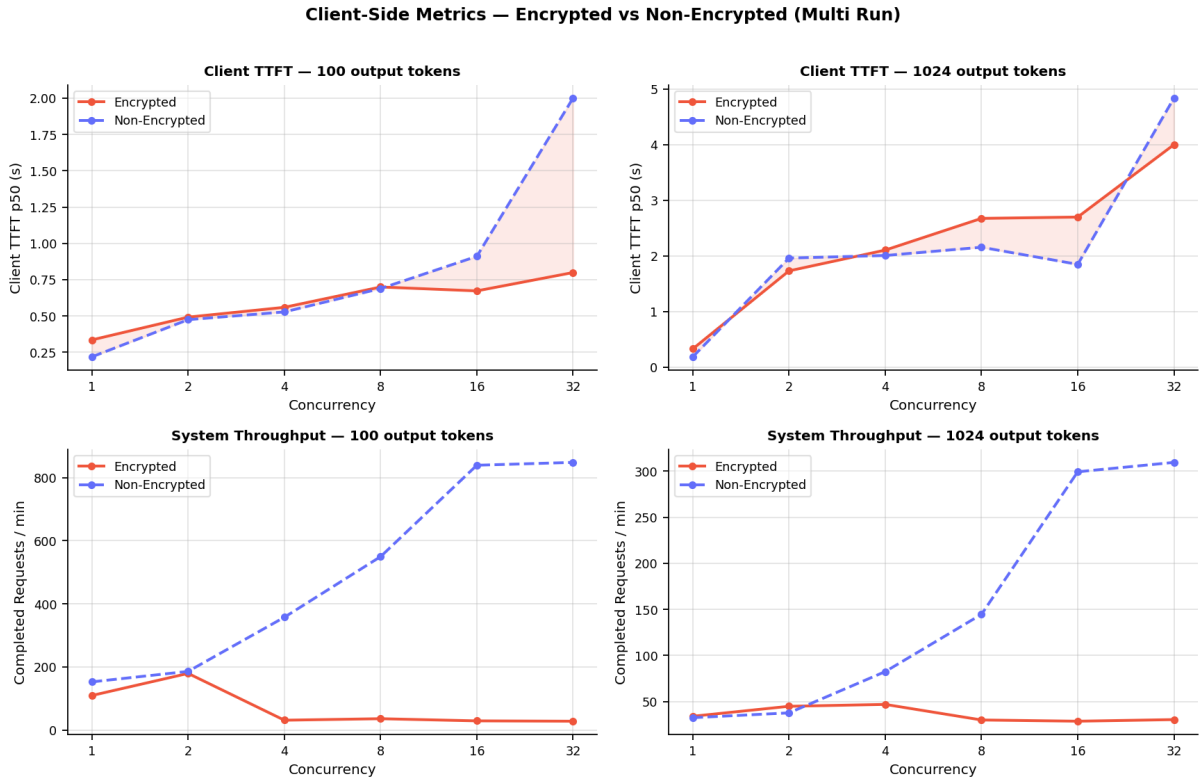

Figure 5: Client TTFT p50 (top) and completed requests per minute (bottom) for 100 and 1,024 output tokens. Multi Run (10x). At concurrency ≥ 16 (100 tokens), encrypted TTFT drops below non-encrypted due to the TEE de-batching benefit on server prefill time.

■ Warning: System throughput gap widens at high concurrency (Encrypted)

Non-Encrypted throughput scales near-linearly with concurrency, reaching ~848 req/min at concurrency 32 (100 output tokens). Encrypted throughput peaks at concurrency 2 (~180 req/min) and falls to ~28 req/min at concurrency 32 — a ~30× gap. This collapse is driven by the TEE queue building up (>8 cores) and requests experiencing progressively longer network waits. At concurrency >8, error rates reach 11–19%, further reducing effective throughput. Note: despite lower per-request TTFT at high concurrency for 100-token responses, the overall system throughput (req/min) remains far lower for the encrypted path due to TEE queueing and errors.

6. Summary Data Table (Multi Run)

All latency values in seconds unless noted. TEE % = (network + encryption + decryption) / encrypted client TTFT × 100.

Out Tokens	Conc	Server TTFT Plain	Server TTFT Enc	Client TTFT Plain	Client TTFT Enc	Network (ms)	Encrypt (ms)	Decrypt (ms)	TEE % of TTFT	Error Rate
100	1	0.091	0.091	0.218	0.335	425	9.8	3.72	131%	0%
100	4	0.161	0.162	0.527	0.559	600	7.9	1.12	109%	0%
100	8	0.311	0.162	0.689	0.699	638	7.0	1.07	92%	0%
100	16	0.311	0.165	0.910	0.672	756	5.2	0.77	113%	11%
100	32	0.312	0.164	2.001	0.799	803	5.5	0.71	101%	19%
1024	1	0.092	0.091	0.183	0.326	1608	9.1	10.88	499%	0%
1024	4	0.161	0.163	2.006	2.103	2735	9.4	7.11	131%	0%
1024	8	0.312	0.161	2.154	2.674	2266	8.9	6.42	85%	1%
1024	16	0.311	0.311	1.847	2.697	2679	8.1	5.16	100%	9%
1024	32	0.311	0.167	4.838	4.000	3149	5.9	5.19	79%	18%

7. Conclusions

• **TEE adds zero server inference overhead — confirmed.** With equivalent endpoint configurations, server TTFT and tok/s are nearly identical at concurrency 1–4 (~0.091 s and ~585–620 tok/s respectively). The TEE de-batching effect is also confirmed: at concurrency 8–32, encrypted server TTFT stays flat at ~0.163 s while non-encrypted rises to ~0.311 s. At concurrency ≥ 16 with short responses, this server-side benefit can outweigh the TEE network overhead entirely.

• **The overhead is entirely network-driven.** The extra network hop through the TEE accounts for 85–97% of the additional client-visible latency. Encryption (~5–10 ms) and decryption (~0.1–4 ms cumulative) are negligible.

• **Network latency is predictable at low concurrency.** Below TEE capacity (concurrency ≤ 8), the added latency is roughly constant per round-trip and scales linearly with output token count — consistent with the TEE team's claims.

• **TEE queue saturation is the critical scalability limit.** At concurrency > 8, the 8-core TEE begins queuing requests, causing latency to grow non-linearly and error rates to increase. Production deployments should stay at or below the TEE core count for reliable performance.

• **Per-user latency overhead at concurrency 1:** Client TTFT increases by ~0.19 s (100 output tokens) to ~0.17 s (1,024 output tokens) in Multi Run — relatively modest for a confidential computing solution.

Recommendations

• **Reduce TEE network latency:** Co-locate the TEE as close as possible to both the client and the SambaNova inference server to minimise round-trip times.

• **Respect TEE core limit:** Keep concurrent request load at or below 8 to avoid queue saturation. Above this point latency grows non-linearly and errors appear.

• **No need to optimise crypto:** Encryption and decryption are already fast enough that further optimisation would have negligible user-visible impact.

• **Scale horizontally:** For higher throughput with the TEE, scale the number of TEE instances rather than increasing concurrency per instance beyond 8.